

Lec 48 Hand on on forward Diffusion process

Lec 48 Hand on on forward Diffusion process

Lecture	48 / 63 + intro
Duration	20:05
Captions	English (auto-generated)
Words	2,792
Video	wXBxC49sNmk
Playlist	Fundamentals of Generative AI and LLMs

Auto-generated YouTube captions, lightly cleaned (removed [music] tags, grouped into paragraphs). These are **verbatim lecture captions**, not edited notes.

Transcript

[00:00] Hello everyone. This is Devarpan. Today we'll be covering the part one of the tutorial on diffusion models that are already covered as part of the theory classes. Today we'll be going through the first part of diffusion model which is primarily focused on the forward process of diffusion and the backward diffusion process will be covered in the next tutorial on diffusion models. So let's get started. So coming to the diffusion models, the intuition and the forward process specifically. This tutorial covers primarily the lectures that are covered in week seven. And the and the hands-on material covers the primary questions about diffusion models that are why diffusion models forward diffusion by adding Gaussian noise.

[00:56] It covers the linear and cosine variance schedules of the noise addition to the signal or the image. The denoising intuition, Markov chain interpretation of diffusion model diffusion process. Closed form sampling at any time step and how it compares to the iterative process. Signal to noise ratio, noise prediction objective and training a small time step conditioned CNN on the MNIST data set. Okay. Coming to the learning objective of the diffusion model.

[01:36] Here is a set of a set of imports that you need to perform and then setting the device to GPU and making torch related imports and so on. Coming to the intuition

of diffusion model, a diffusion model separates generation into two processes. First is the forward process, which starts with the original image denoted as x_0 or original signal. And then in iterative process x_1 , x_2 , and so on till x_T , it converts it to a Gaussian noise in small steps.

[02:12] Small amount of Gaussian noise are added. And in the reverse process, you intend to start with x_T , which is um a normalized Gaussian noise, and then you intend to come back or predict the original image that x_0 is. So, neural network is required to learn the reverse diffusion process. That is x_T from x_T to x_0 . Um So, coming to the forward um transition as we talked about. So, Q uh uh distribution, which is distribution of next step given the previous step step, that is $Q(x_{t+1} | x_t)$ given x_t minus one, that is um Gaussian um with mean of um square root of $1 - \beta_t$ times x_t minus one and variance of β_t times the identity matrix, that is the covariance matrix. So, this uh how this came and derivation of this was covered

[03:15] as part of the theory. Now, you define um α_t as $1 - \beta_t$, and you can define $\bar{\alpha}_t$ as pi of like multiplication of all alphas uh till that step. And then any noisy state, you can sample, uh which is denoted as x_t as and there is a closed form expression that you can derive, uh which uh which is square root of $\bar{\alpha}_t x_0$ plus the square root of $1 - \bar{\alpha}_t$ times epsilon. That is the expression of any intermediate step of the diffusion process with respect to the original signal we started with, that is x_0 .

[03:58] Okay. Now coming to the variance schedules that uh as you have noticed we start with the original image and go to a Gaussian noise. Now these steps can be varied in a linear or a non-linear fashion. So we discuss one of the two most widely used uh variance schedules, which is linear, which is simple, versus um the uh cosine schedule, right? So so let's start with T let's set T equal to 1,000, that is 1,000 number of steps will be there starting from the original image to the noisy image for a noise Gaussian noise.

[04:42] And uh you can define the linear beta schedule schedule or a cosine derived beta schedule, uh which are implemented. Yeah? Now you can see that both ranges starting and end points are same. It goes from zero to one. Okay? And uh now you can essentially plot these schedules and you can see that see how these steps varies. You can see that the linear beta stays same from the time starting time step to end time step. It stays uniform to one upon thousand, obviously. And uh the cosine beta, it starts with a smaller value and then slowly increases with the increasing time steps.

[05:29] Yeah? Uh Okay. Now Now you can essentially also plot the SNR, that is what is the fraction of original signal that is retained when you start from that starting time step and the original image or original signal and slowly go towards the Gaussian noise. You can see that um because cosine takes smaller steps initially and then suddenly increases the um steps. So, cosine schedule has a um So, compared to

linear, cosine has a more uniform decline of the uh fraction of the original signal.

[06:10] That's why cosine is preferred. Although linear uh takes the uh s- takes uniform steps um across the time stamps. um So, it has a initially slowly degrading uh signal and then it suddenly degrades. Whereas cosine has a more uniform rate of decline. So, how what is the quantity of the original image that remains is defined by the αt bar that we defined earlier. Now, let's uh examine the forward diffusion process and implement it uh on a real image. Okay. So, we uh take a image um We simply load a image using SK image.

[06:50] Uh an astronaut image which is commonly used in these experiments you might have seen. And if you display the image, it looks like this. Yeah, that's the original image. And uh you can now um time stamp specific coefficients you need to calculate as we have defined already the the alpha and betas. So, you define extract function which takes values time steps and target shape. And you also define a function called Q sample which samples a noisy image directly from the $Q \times t$ given x zero. So, this is the marginal distribution of any intermediate step given the original image or original signal.

[07:35] Okay. um Now, for the for just visualization, we need to see how the intermediate steps look like. We define these visual time steps as some intermediate steps like 0, 50, 100, like that till 999, which is the last. Okay, now you if you try to visualize it, it looks like this. You start with the zeroth time step and then you go to 999 time step and you can see that um Uh you can see that the image slowly converts to the Gaussian noise at the end of the process. And intermediate steps are kind of like the image slowly getting more noisier and noisier.

[08:25] Now, we can you can also visualize how uh how these two different variance schedulers, which are like linear and cosine schedulers, um different from each other. So, in the first row, you can see that the initially the image gets um gets suddenly so noisy and and within 500 time step, it's almost like the close to the Gaussian noise. Whereas, as we had shown you through the graph or plot before that uh the original image slowly decline into a Gaussian noise in more uniform fashion in the cosine [clears throat] scheduler cosine variance schedule um that visualization you can see in the bottom row where the image much more uniformly converts to the Gaussian noise.

[09:18] Okay, now comes a very important part of it uh part of diffusion models forward process, which is Markov chain interpretation. The forward diffusion process can be interpreted as a Markov chain. What do you mean by Markov chain? So, Markov chain has this property that the next step only depends on the previous step. So, so um this uh this is an important interpretation of forward diffusion process that to get the next noisier image, you only need the previous noisier image. Okay. So, just to define mathematically, the distribution marginal distribution of X_T given X_{T-1}

1 X_{T-2} till X_0 , that is nothing but Q of X_T given X_{T-1} . So, given X_{T-1} , to get the distribution of X_T , you don't need X_{T-2} till X_0 anymore.

[10:09] Once X_{T-1} is known, you can get the distribution of X_T . So, let's implement uh the Markov forward transitions um using beta D and alpha D formulations that we already had shown the formulation for. And here is a visualization of the Markov process. So, you start with T equal to 0 and T T equal to 999, and then you can see that similarly this similar to what our uh forward diffusion process is supposed to give, um similarly this image slowly gets converted to the Gaussian noise.

[10:45] Now, now the important question you will have is um how they compare. Right? So, so we had a closed form expression of Q X_{T-1} given X_0 , and and you had a forward diffusion process based on that. But then then based on the Markov chain interpretation of the diffusion process, you can you can also get a uh uh uh original image to noise conversion steps. Now, how they differ from each other? Um what is the variance? So, we essentially um take the iterative and direct methods generally produce different individual samples because they are different random draws. They nevertheless represent the same marginal distribution. The following Monte Carlo experiment compares their empirical mean and standard deviation. Uh so, we define

[11:35] um the uh clean signal, which is X_0 , which is simply a scalar let us take. And then uh you define the target time step to fire 500, number of samples to be 20,000. Iterative um process you can define and compare the closed form versus the iterative process of using Markov chain. So, you can see that the um mean and standard deviation that are obtained are pretty close to each other. That that uh lead that validates our um previous theoretical finding and understanding that Markov chain modeling is enough when you do closed form sampling in forward diffusion process.

[12:17] Now, you can define signal-to-noise ratio um defined as $\alpha \bar{t}$ divided by $1 - \alpha \bar{t}$ at time step t . So, signal-to-noise ratio should be uh high at the initial step because there is only signal and no noise. And as you go forward and you uh signal reduces and noise increases, so SNR should go down. And at the final step, there is only noise. There is no signal. Um So, if you plot it, you can see that that actually happens. [clears throat] Um both in case of linear and cosine schedule, the SNR monotonically keeps decreasing. Only variance is how it decreases. Another limitation of the linear scheduler, which was much simpler, but um it we saw some limitation of it earlier and this is here is another that it um

[13:11] it doesn't degrade your SNR to that low value where cosine scheduler can degrade you degrade the process to. So, ideally our our you know, um uh Uh, our our desired process is that S and N should be as high as possible at the starting and it should be as low as possible at the end. So, cosine schedule gives a much better range uh, from that aspect. And here we come to the noise prediction objective, which is which um, is an important part of the next part, which is reverse diffusion process.

So, we ask this question that um, you have a clean image X_0 and you have random time step T and you sample the Gaussian noise ϵ and you constructed X_T , which is a intermediate teeth stiff um, of the

[14:11] forward diffusion process. Now, given X_T and T uh, to a neural network can you predict the noise ϵ ? Because this is not a linear process um, and you need to use a neural network, but can it predict it? Can it Can it predict the noise? Given these arguments. So, the simplified objective that you essentially use as a training objective or loss to minimize it using gradient descent is expectation of ϵ minus ϵ_θ . So, ϵ_θ represents θ represents the neural network parameters. So, you you essentially are trying to learn an ϵ using a neural network θ given the arguments X_T and T .

[14:52] Can you predict ϵ ? So, squared error between them. And you train a simple MNIST noise predictor using it. Okay. So, you define the transforms and train test splits on that and you just um, obtain the train and test data for your neural network training. Training examples are 12,000 and validation examples are 2,000. And here is set of clean MNIST training images which we used to um create the data set. And uh we define a CNN which takes the images as uh which takes the image and um tries to predict the noise as we have already defined. Um so we define the neural network for that and uh here are the number of parameters which is like 124,000 order um parameters.

[15:56] And then you define the batches and enable the training for it and the evaluation function. Yeah. So what you obtain is that um the mean squared error that you already defined as L simple keeps reducing with the training steps. And validation MSE also reduces. And what you observe is that the mean squared error keeps reducing with more optimizing steps. That means that shows that uh you can quite well approximate the noise.

[16:29] Um given the arguments of time step and the X_T . X_0 and T . So So let us So let us assume that you have as you have predicted the noise quite well. That is ϵ at θ X_T is is a very good approximation of ϵ that you have already done. So So under that assumption you should be able to reconstruct your original image. So that's a cross validation of how your training is apart from the loss which is a crude way of measuring. So if you have predicted predicted the noise well then um if you subtract that predicted noise from the noisy image you should get back your original image.

[17:18] So, if you try to visualize it, um then you can see that um this first row and last row compares. Uh so, first row where the original image is and last row are the reconstructed images by subtracting the learned noise. You can see that uh you here is a this was a very small scale implementation of it. This and also one iteration of it. Actual reverse diffusion process will use lot many steps and will have much

more uh predictive power, but you can approximately see that you kind of get back your original images now. Although not of great quality, but here here was just a toy example of how this works.

[18:02] How this concept works. We'll go into this much deeper in the next part of diffusion model. Um yeah. So, uh So, yeah. A decreasing noise prediction loss does not mean that the model has all can already generate high-quality images uh that you can see. And uh a complete DDPM will still actually require reverse process equations, iterative sampling, um rather than this one-step solution, and a stronger unit architecture rather than a simple CNN, uh longer training, and um uh optionally some conditioning and guidance, which we'll discuss in the next part of the tutorial. Yeah. So, they will be uh taken care And uh in case you wanted to want to um get your hands dirty further on this uh and feel

[18:52] looks interesting to you, here are a couple of exercises I have noted down for your um own exploration. Okay. So, I'll um finish this uh today's tutorial with uh just a summary. Uh forward diffusion gradually destroys the image structure with Gaussian noise. This process is Markovian. A variance schedule determines how rapidly the signal is removed. Yeah, and the closed form equation enables efficient random time step training.

[19:25] The signal-to-noise ratio decreases with time step. Um the network receives both X_T and T when it is trying to predict um the noise for a particular time step T . A basic DDPM often learns to predict the added Gaussian noise using the MSE error that it tries to minimize using a uh neural network. Full reverse diffusion will be uh covered in the next part of the tutorial. And um uh that's it for uh this tutorial. And uh thank you.